



Integrating Programming Skills into Engineering Education in the AI-Driven World

Shuo Yan^(a); Lasantha Meegahapola^(a); Guanliang Chen^(b)

RMIT University^(a), Monash University^(b)

Corresponding Author Email: shuo.yan@rmit.edu.au

CONTEXT

With the rapid expansion of AI technologies, programming skills have become increasingly vital in the global education and technology landscape. Modern programming languages, with their simplicity and powerful libraries, enable engineers to tackle complex problems involving large datasets, intricate systems, and uncertainty. As AI-driven and data-centric projects continue to grow, programming proficiency is becoming indispensable. However, most engineering programs lack a structured approach to addressing this urgent need, leaving graduates underprepared with the necessary coding skills.

PURPOSE OR GOAL

This digest aims to embed AI related programming into the engineering program through several scholarly approaches while maintaining the existing program structure, using Python as the medium. The objective is to support students, especially those with little to no prior programming experience, in progressively developing their programming skills. They will transition from solving simple engineering problems to addressing challenging problems in various electrical engineering domains.

APPROACH OR METHODOLOGY/METHODS

The program design will align with the progressive learning structure of the engineering curriculum. Firstly, lecture notes will be distributed to students for self-learning in the first year, supported by drop-in sessions. Secondly, AI-related coding knowledge will be integrated into relevant core courses through flipped classrooms, enabling students to apply programming skills more practically. Taking the core engineering course as a trial, the design will combine the programming skill with key topics such as linear regression and Newton-Raphson method. A comparative study will be conducted to assess the performance of students with and without programming-integrated learning.

ACTUAL OR ANTICIPATED OUTCOMES

The project aims to: (i) enhance engineering graduates' ability to apply programming techniques to real-world scientific and engineering challenges in an AI-driven world; (ii) foster a problem-solving mindset that leverages programming for complex engineering issues, particularly through AI-based toolsets such as neural networks; and (iii) improve graduates' competitiveness by equipping them with essential knowledge in AI, machine learning, and their applications in engineering.

CONCLUSIONS/RECOMMENDATIONS/SUMMARY

This paper introduces a progressive approach to integrate coding skills across the engineering degree program without making substantial changes. The progressive integration with core engineering courses is effective to address the restriction of time commitment and problem-oriented learning. The design will fill the gap in the contemporary programming training of engineering graduates.

KEYWORDS

Engineering program, AI-driving projects, programming skills.

Introduction

With the rapid expansion of artificial intelligence (AI) technologies, programming skills have become increasingly important in the global education and technology landscape. This trending requirement on engineers is highlighted in the Body of Knowledge Framework for Future Professional Engineers and Technologist, as shown in Figure 1 (Shan, 2024), where the digitalisation, cyber security, and information technology are the key component in the engineering knowledge spectrum. Driven by the simplicity and powerful libraries of contemporary programming languages, engineering problems increasingly associate with large volumes of data and higher system complexity and uncertainty. In the recent engineering practices, there has been a strong recommendation to embed programming skills into the engineering program curriculum. These requirements are mostly from the professional engineering bodies, such as the Hong Kong Institute of Engineers (HKIE), and the Engineer Australia (EA).

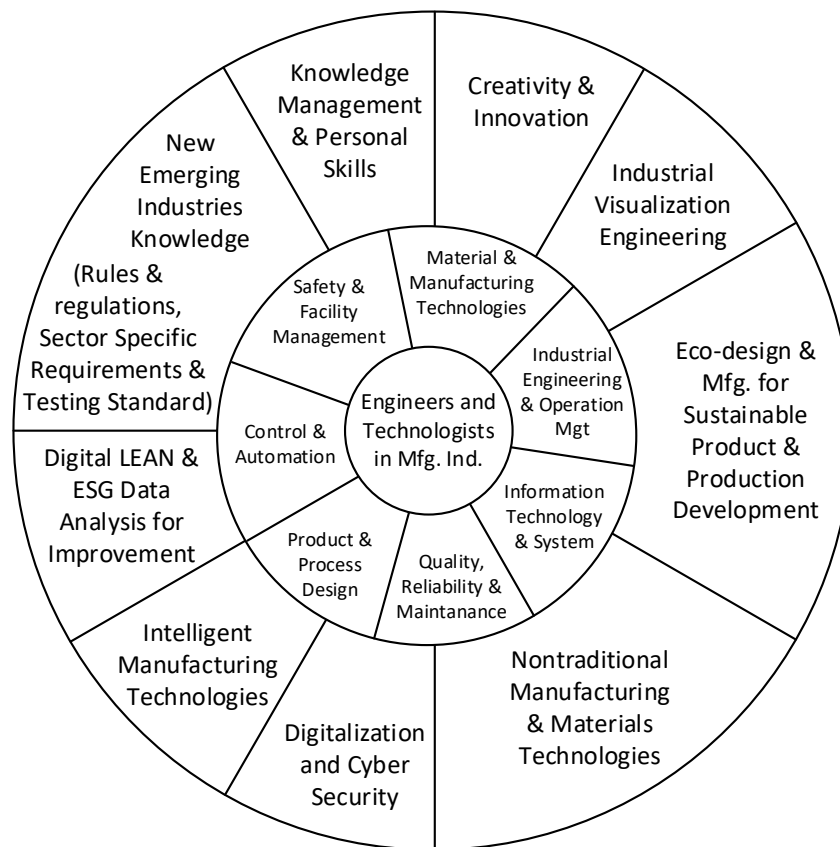


Figure 1. The “Body of Knowledge (BoK2)” framework for future professional engineers and technologists by HKIE (Shan, 2024).

Despite these recommendations, the current program structure provides limited opportunities for students to develop systematic programming skills. Although a few supplementary modules have been introduced, students are largely expected to conduct self-learning, which often proves inadequate for meeting the challenges of contemporary engineering practice (Licorish et al., 2022). Addressing this gap requires a structured approach to embedding programming into the curriculum.

This project proposes a scholarly, phased integration of Python programming within the engineering program while maintaining the existing course structure. The goal is to support students, especially those with limited or no programming experience, in progressively developing their coding skills. Students will move from solving basic engineering problems to addressing complex, real-world challenges across various electrical engineering domains. Ultimately, they will be equipped to design practical engineering solutions using structured programming approaches.

The proposed strategy begins with providing self-learning materials and lecture notes in the first year, complemented by drop-in sessions to assist with questions and challenges. In the second stage, programming tasks will then be linked to core courses. Topics that cover general numerical knowledge sets will serve as practical programming exercises within this course. Finally, to support students in their final-year projects, tailored programming toolboxes and support suitable for AI applications will be introduced alongside project supervision. By embedding programming modules in this structured and progressive manner, the project aims to enhance students' ability to apply programming techniques to real-world engineering and AI-driven challenges, improve their competitiveness in the job market.

The first stage of this project was successfully piloted within the Electrical Engineering program at RMIT University, where dedicated programming workshops were delivered to two student cohorts in 2024 and 2025. These workshops focused on fundamental programming concepts while emphasising key aspects relevant to AI-driven tasks, such as data handling, algorithm development, and problem-solving techniques. Students were required to participate in the workshops and complete in-class exercises designed to build their confidence in applying programming skills to future project work. To evaluate the effectiveness of the workshops, we accounted the number of projects that incorporated AI-related or machine learning (ML) programming elements and calculated the percentage of these projects. The statistics in years 2020 to 2024 are monitored and compared. In addition, student outcomes were assessed by analysing the results of the technical design. These observations provided valuable insights into how early exposure to programming and AI concepts can influence students' engagement with advanced engineering applications.

The findings suggest that early and consistent engagement with programming training significantly enhances students' willingness and confidence to apply these skills in solving real-world engineering problems and completing design projects (Abesadze & Nozadze 2020). Building on these results, the next phase of the project will integrate programming and AI techniques into core disciplinary courses by developing tutorials and assignments that require the practical application of these skills. Additionally, a broader emphasis will be placed on incorporating programming and AI in design projects. Future research will further explore how this systematic integration can strengthen the value of engineering degree programs. By embracing emerging technological skills, this approach not only equips students with work-ready skills but also fosters greater interest in engineering careers among prospective students and the broader community.

Real World Challenge

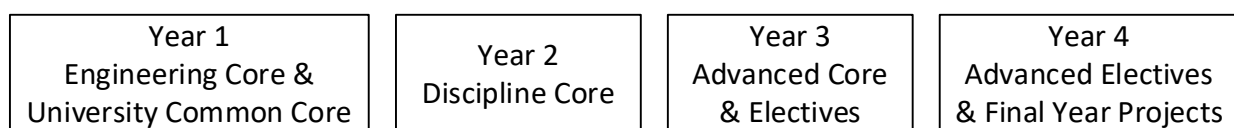


Figure 2. Typical four-year structure of the engineering program.

With the rapid increase of AI technologies, the demand for engineering graduates with strong programming skills has increased substantially. Industry partners and professional accreditation bodies have consistently emphasized the need to strengthen the programming proficiency of engineering graduates (Bhoga et al., 2024). This reflects a technological shift where data-driven analysis and AI-based applications are becoming fundamental to engineering practice. However, the current program structure lacks a systematic approach to address this need and adequately prepare students with AI-related programming skills. As illustrated in Figure 2, typical engineering curricula focus heavily on delivering theoretical knowledge, with limited integration of AI technologies as a tool for practical problem-solving. While introducing dedicated fundamental programming courses might enable students to acquire technical skills within a short, intensive period, such standalone courses often fail to ensure continuity. Once the course ends, students typically have few opportunities for further application or reinforcement of these skills. This gap is evident in the low percentage of design projects that incorporate programming techniques or AI applications in years from 2020 to 2023, which oscillates round 21%, based on the data collected in the RMIT electrical engineering in Hong

Kong. Most projects continue to follow conventional engineering design processes and rely on traditional analytical methods. Consequently, treating AI and programming as isolated, one-off subjects does little to cultivate the habit of applying these tools to real-world engineering challenges.

To effectively embed AI-oriented problem-solving into engineering education, programming and AI applications must be progressively integrated across the curriculum. This continuous reinforcement allows students to apply their skills in various course contexts and project work, fostering sustained engagement and practical competence. In line with this philosophy, the project adopts a scalable approach by embedding a series of targeted teaching and learning activities throughout the program cycle. These activities are designed to build students' foundational programming knowledge, enhance their problem-solving capabilities, and strengthen their ability to apply AI in project-based contexts. The initiative employs engaging methodologies such as drop-in sessions, flipped classrooms, and project-based learning (Brusilovsky et al., 2018), ensuring that students receive ongoing support through consultations and project supervision. Besides, this strategy provides a practical pathway to integrate programming and AI application skills across the students' learning journey without requiring substantial changes to the existing program structure, addressing common challenges like limited time for dedicated coding training (Wende et al., 2020).

Existing Solutions

The traditional approach to integrating programming into engineering education often relies on microcontroller units such as Arduino and Raspberry Pi (Nazarov et al., 2021). This method, commonly incorporated into digital fundamentals or microprocessor courses, focuses on task-oriented code development and peripheral device control, hence failing to provide a comprehensive view on the foundation in programming or computational thinking.

Another approach in engineering education is the integration of computational skills through discipline-specific frameworks. One example is in civil engineering, where course assignments consist of coding requirements (Abell et al., 2024). However, this approach lacks a structured progression in developing programming skills. The abrupt introduction of programming tasks without foundational training creates learning inconsistencies, particularly for students with little or no prior coding experience, making it difficult for them to achieve the expected learning outcomes.

Additionally, game-based and problem-based learning have been explored as alternative approaches to teaching programming (Topalli & Cagiltay, 2018). While these methods can enhance engagement and motivation, they often lack a direct connection to disciplinary knowledge. As a result, the transition from game-based learning to engineering applications tends to be ineffective, limiting the immediate applicability of programming skills in real-world engineering contexts.

More importantly, existing research has largely overlooked the integration of machine learning (ML) and artificial intelligence (AI) as core components of programming education. Given the growing role of AI in science and engineering, its exclusion represents a critical gap in current curricula (Medeiros et al., 2018).

To address these challenges, this project proposes a problem-oriented, discipline-integrated approach to programming education for engineering students. The objective is to develop a structured, progressive study module that is closely aligned with engineering disciplines and rooted in real-world problem-solving. Furthermore, recognizing the increasing importance of AI technologies, the project will incorporate AI-focused elements to better prepare students for future industry demands.

Methodology

Based on the Bloom's taxonomy shown in Figure 3, the methodology to integrate the AI-oriented programming technologies is designed with three progressive stages:

Stage 1 (Learning Stage) focuses on foundational knowledge building through self-directed study, complemented by weekly drop-in support sessions. Students will engage with core programming

concepts — such as data structures, functions, and compiling platforms — using structured lecture notes. Learning outcomes will be reinforced through quizzes and assignments. A student survey will assess the perceived usefulness of the module and its relevance to future study and career aspirations. Engagement will be monitored via attendance at support sessions and assignment submission rates.

Stage 2 (Applying Stage) integrates programming into core engineering courses, enabling students to apply coding skills to authentic engineering problems, such as stability assessment via eigenvalue analysis, and curve fitting using linear regression. Student feedback on the coding-based approach and its impact on assignments and design tasks will be collected through surveys. The effectiveness of this integration will be further evaluated by comparing final grades and performance outcomes with cohorts from previous offerings without emphasizing the AI programming.

Stage 3 (Designing Stage) culminates in a capstone design project, where students tackle complex engineering challenges using advanced coding techniques. Targeted lectures will introduce contemporary applications, for example, facial recognition, to expand students' problem-solving toolkit. Project assessments will evaluate the extent to which programming skills enhanced design outcomes, with particular attention to the proportion of projects featuring substantial coding components.

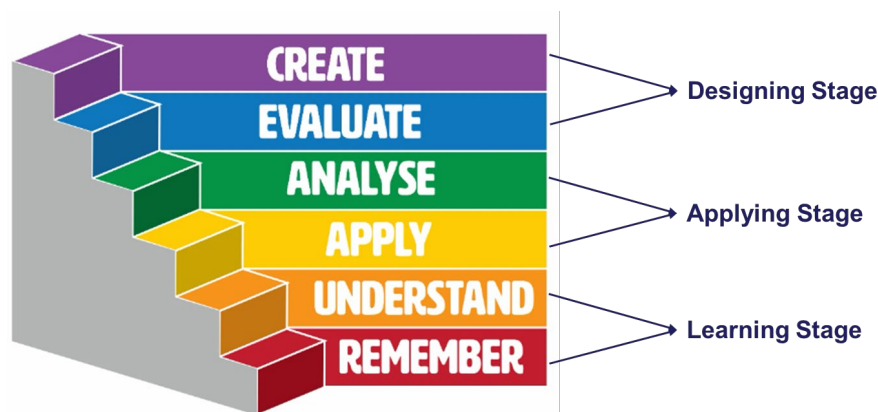


Figure 3. Bloom's taxonomy as the methodology design framework.

Project Design

This project aims to design a staged teaching and learning model, aligned with different phases of the program cycle. The learning outcomes will be evaluated through both students' performance and their perceptions. Specifically, surveys will monitor students' evolving mindset and readiness to use AI programming in coursework and project design. The flowchart of the project design is shown in Figure 5.

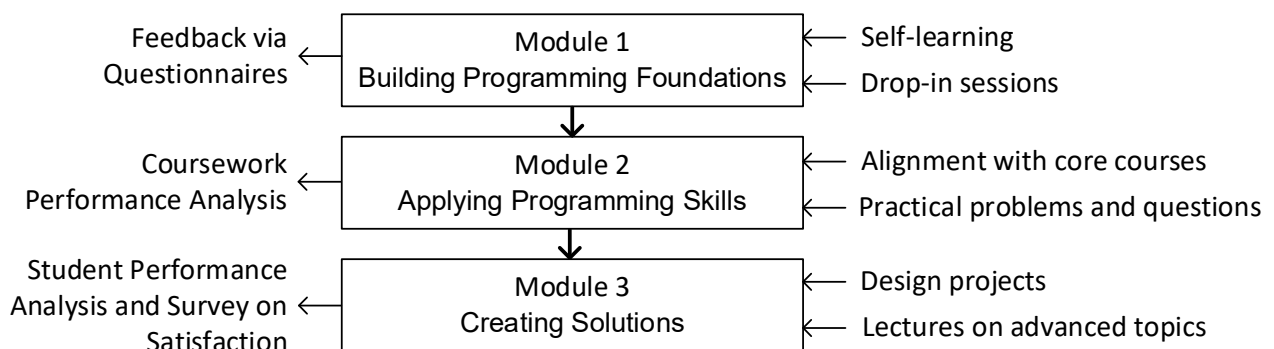


Figure 5. Flowchart of project methodology.

Module 1: Building Programming Foundations through Self-Learning and Drop-in Support

Approach: Lecture notes will be developed for first-year students to support self-directed learning. Weekly drop-in sessions will aid with coding challenges and questions. This format aims to build students' understanding of fundamental programming concepts and familiarize them with coding platforms. Monthly quizzes and coding assignments will reinforce learning and offer continuous feedback on progress.

Objective: To progressively develop students' fundamental programming skills over the first academic year, reducing fear and unfamiliarity with coding languages, syntax, and platforms. Through regular practice, students will gain confidence in applying basic programming to common numerical, graphical, and procedural tasks.

Evaluation: Student feedback will be gathered via questionnaires focused on perceptions of programming, its role in problem-solving, and its impact on their learning experience.

Module 2: Applying Programming Skills through Problem-Oriented Assignments

Approach: Programming will be embedded within core engineering courses. Students will undertake small-scale projects using coding tools to address specific engineering problems, supported by guided assignments and drop-in sessions. These assignments will assess students' ability to integrate course knowledge with programming techniques as part of their regular coursework.

Objective: To encourage students to actively apply programming skills in solving engineering problems, fostering a mindset where coding is seen as a necessary and practical tool. Student performance will be benchmarked against previous non-programming cohorts.

Evaluation: Coursework performance, particularly on tasks emphasizing programming, will be recorded and analysed. Student perceptions of how programming supports their problem-solving will be captured through targeted surveys.

Module 3: Creating Solutions with Programming Skills through Project-Based Learning

Approach: Final-year capstone projects will incorporate significant programming components, challenging students to solve complex, real-world engineering problems. To support this, guest lectures will showcase contemporary engineering applications of programming, particularly AI-driven solutions. Projects will be evaluated on design quality, technical implementation, and presentation, with emphasis on the effective use of coding.

Objective: To engage students in realistic design projects that integrate advanced programming and AI applications, fostering deeper problem-solving skills and professional readiness.

Evaluation: Project relevance, use of programming and AI techniques, and student performance will be systematically reviewed. Post-project surveys will assess students' sense of achievement, skill development, and satisfaction with their learning experience.

Outcomes

The outcome of the first stage was measured by tracking the proportion of students who incorporated AI technologies into their capstone design projects over two cohorts in years 2024 and 2025, which are 24.6% and 28.3%, respectively. These results are based on the statistics of engineering Capstone projects of the electrical engineering program at RMIT HK. These percentages were compared against previous-year cohorts who had not experienced the progressive teaching and learning approach. The comparison serves as an initial indicator of how sustained exposure to AI-related programming influences students' willingness to adopt such technologies in solving real-world problems.

While tentative, this measure provides insight into the effectiveness of embedding consistent and progressive AI-related programming instruction throughout the program. It reflects the gradual development of students' skills, confidence, and competence, which critical attributes for their future

engineering careers. Over time, this approach aims to foster a mindset shift within the engineering community, positioning AI-related programming as integral tools for addressing complex, real-life engineering problems.

Limitation and Future Research

The current research remains in a pilot phase with a limited sample size, which restricts a comprehensive understanding of the proposed strategy's effectiveness. The current evaluation model, which explores students' perceptions of using AI-related programming skills, is still largely intuitive. To obtain accurate and reliable results, it is essential to develop professionally designed and systematic questionnaires. This process may benefit from incorporating psychological studies to assess changes in students' mindsets at different stages of learning.

Furthermore, educators represent a critical domain for investigation. Their perceptions and understanding can offer valuable insights for refining the implementation of teaching modules and materials. In particular, educators' views on how progressive teaching of AI-related programming skills supports student academic performance could serve as a key indicator of the project's effectiveness.

Therefore, future research will aim to enhance the evaluation framework by scientifically examining the psychological development of students throughout the learning stages and by including educators as an additional research cohort.

Conclusion

This paper presents a framework for integrating AI-relevant programming techniques throughout the lifecycle of engineering programs. The objective is to establish a consistent and progressive training pathway that enables students to build foundational programming skills, apply these skills in course assignments, and leverage them in complex design projects. Student performance and perceptions will be monitored to evaluate the impact of this training model on their mindset and ability to apply AI-related programming in practical engineering problems. Specifically, questionnaires will track psychological evolution, while selected assignments will assess performance outcomes. The findings aim to demonstrate that continuous programming training is essential for reshaping the knowledge base of future engineering professionals, thereby facilitating a smoother transition of skills required for addressing real-world engineering challenges.

References

- Shan R. (2024). Hong Kong's path to future manufacturing: Harnessing New Quality Productive Forces and New Industrialisation. *The Journal of The Hong Kong Institute of Engineers*, 52, Cover Story.
- Licorish, S. A., Galster, M., Kapitsaki, G. M., & Tahir, A. (2022). Understanding students' software development projects: effort, performance, satisfaction, skills and their relation to the adequacy of outcomes developed. *The Journal of Systems and Software* (186), 186.
- Abesadze, S., & Nozadze, D. (2020). Make 21st century education: the importance of teaching programming in schools. *International Journal of Learning and Teaching*, 158-163.
- Bhoga, U., G., V. M., & A., D. B. (2024). Coding club's impact on engineering education. *Journal of Engineering Education Transformations*, 37.
- Brusilovsky, P., Malmi, L., Hosseini, R., et al. (2018). An integrated practice system for learning programming in Python: Design and evaluation. *Research and Practice in Technology Enhanced Learning*, 13, Article 18.
- Wende, M., Giese, T., Bulut, S., & Anderl, R. (2020). Framework of an active learning Python curriculum for first year mechanical engineering students. *In Proceedings of the 2020 IEEE Global Engineering Education Conference (EDUCON)* (pp. 1193–1200). IEEE.

- Nazarov, S., & Jumayev, B. (2021). Programming-based laboratory assignments for undergraduate students of power engineering fields. *In Proceedings of the 22nd International Conference on Computer Systems and Technologies*.
- Abell, J. A., Moreno-Casas, P. A., & Recabarren, M. (2024). Integrating advanced computational skills into engineering education: A discipline-based approach. *Computer Applications in Engineering Education*, 32(6), e22784.
- Topalli, D., & Cagiltay, N. E. (2018). Improving programming skills in engineering education through problem-based game projects with Scratch. *Computers & Education*, 120, 64–74.
- Medeiros, R. P., Ramalho, G. L., & Falcão, T. P. (2018). A systematic literature review on teaching and learning introductory programming in higher education. *IEEE Transactions on Education*, 62(2), 77–90.

Copyright statement

Copyright © 2025 Names of authors: The authors assign to the Australasian Association for Engineering Education (AAEE) and educational non-profit institutions a non-exclusive licence to use this document for personal use and in courses of instruction provided that the article is used in full and this copyright statement is reproduced. The authors also grant a non-exclusive licence to AAEE to publish this document in full on the World Wide Web (prime sites and mirrors), on Memory Sticks, and in printed form within the AAEE 2025 proceedings. Any other usage is prohibited without the express permission of the authors.